AI Project Report

David Curral 64530
Daniel Gonzalez 58961

Este projecto foi feito com base nos conteúdos fornecidos para a ficha de Decision Trees.

Existem vários notebooks, com funcionalidades diferentes. Decidimos separar para facilitar o trabalho e a compreensão.

1º Etapa

Em primeiro lugar criámos dois notebooks, um com um carro controlado manualmente por nós e outro com um carro controlado pelo computador. De seguida, convertemos os sensores da ficha (por contacto direto) em sensores tipo sonar, com uma distância variável entre os limites da pista e o carro. Desenvolvemos este sistema de modo a que fosse fácil adicionar ou remover sensores, tendo somente que adicionar as coordenadas/direção e o nome dos mesmos. Assim que o carro ficou pronto, reproduzimos a lógica no carro controlado por nós. Este tinha ainda acrescido a funcionalidade de escrever num ficheiro csv a distância dos sensores e as ações tomadas naquele instante, todos os frames.

```
class DTSimpleCar(AbstractCar):
    IMG = GREEN_CAR
    START_POS = (170, 200)
    SENSOR_POS = [(0, -50),(-30, -40),(30, -40),(0,20),(-50,-5),(50,-5)]
    SENSOR_NAMES = ['north','north_west','north_east','south','west','east']
    #DT = decisionTreeSimple
```

```
def __init__(self, max_vel, rotation_vel, start_pos, start_angle, MAX_RANGE = 100):
    self.max_range = MAX_RANGE  # Maximum radar distance
    self.start_pos = start_pos
    self.start_angle = start_angle
    self.collision_times = []
    self.last_collision_time = None
    self.escape_cooldown = 1500  # milliseconds (1.5s)

    super().__init__(max_vel, rotation_vel)
    self.update_sensors()
    self.x, self.y = start_pos
    self.angle = start_angle
```

Lógica dos sensores.
Basicamente usamos trigonometria para obter a distância entre 2 pontos. Um que é a origem do sensor e o outro a ponta do sensor.
Verificando se a cor do pixel que está a colidir com o sensor é diferente da máscara da pista, podemos adicionar a distância entre o ponto de interseção e a origem do sensor. Depois adicionamos esses valores a uma lista e mais tarde a um ficheiro csv.

```
sensor_dirs = numpy.dot(numpy.array(self.SENSOR_POS), mrot)

for dir_x, dir_y in sensor_dirs:
    norm = math.sqrt(dir_x**2 + dir_y**2)
    if norm == 0: continue
    dir_x /= norm
    dir_y /= norm

    distance = 0
    for i in range(self.max_range):
        sx = int(car_x + dir_x * i)
        sy = int(car_y + dir_y * i)
        if 0 <= sx < WIDTH and 0 <= sy < HEIGHT:
            color = TRACK_MASK.get_at((sx, sy))
            if color != WHITE:
                break
            distance += 1
        else:
            break
    self.sensors.append(distance)  # Store distance
```

No carro controlado pelo computador, decidimos criar uma árvore de decisão à mão para ver como é que o carro se comportava. A árvore e os valores propriamente ditos foram sendo refinados ao longo dos testes. Curiosamente esta árvore é reproduz a melhor condução que conseguimos alcançar, melhor que os modelos treinados posteriormente.

```python
#### Smoother Tree 4 sensors

acc = Action("accelerate")
brake = Action("brake")
rotate_left = Action("rotLeft")
rotate_right = Action("rotRight")

avoid_crash = Less("north_east?", rotate_left, brake, 20)

steer_left = Less("north_east?", rotate_left, acc, 45)

north_west_avoid_crash = Less("north_west?", rotate_right, avoid_crash, 20)

north_west_steer = Less("north_west?", rotate_right, steer_left, 45)

north_check = Less("north?", north_west_avoid_crash, north_west_steer, 30)


speed_check = Less("south?", brake, acc, 20)  # If something is very close behind, stop

final_tree = Less("north?", speed_check, north_check, 30)

decisionTreeSimple = final_tree
```

2º Etapa

Após o carro estar completo, decidimos fazer com que o spawn do veículo acontecesse de forma semi-random. Para tal fomos ver que partes da pista poderiam ser usadas como ponto de partida, e chegámos à conclusão que havia 6 localizações com as características necessárias.

Para gerar tanto o carro como a linha de partida de forma correta, tivemos que calcular as coordenadas dessas posições e rodar, quando necessário a linha e o carro para se enquadrarem corretamente com a pista.

```python
# The finish image intacta
horizontal_finish = pygame.image.load("imgs/finish.png")
vertical_finish = pygame.transform.rotate(horizontal_finish,90) # Negative angle amounts will rotate clockwise.
diagonal1_finish = pygame.transform.rotate(horizontal_finish,45) # Negative angle amounts will rotate clockwise.
diagonal2_finish = pygame.transform.rotate(horizontal_finish,-135) # Negative angle amounts will rotate clockwise.

FINISH_MASK_horizontal = pygame.mask.from_surface(horizontal_finish)
FINISH_MASK_vertical = pygame.mask.from_surface(vertical_finish)
FINISH_MASK_diagonal1 = pygame.mask.from_surface(diagonal1_finish)
FINISH_MASK_diagonal2 = pygame.mask.from_surface(diagonal2_finish)


# Finish line options: [((image, mask), position)]
finish_line_options = [
    ((horizontal_finish, FINISH_MASK_horizontal), (130, 250)),
    ((horizontal_finish, FINISH_MASK_horizontal), (680, 580)),
    ((vertical_finish, FINISH_MASK_vertical), (550, 35)),
    ((vertical_finish, FINISH_MASK_vertical), (530, 220)),
    ((diagonal1_finish, FINISH_MASK_diagonal1), (170, 600)),
    ((diagonal2_finish, FINISH_MASK_diagonal2), (100, 520))]


random_car_location = [((170,200),0),((730,530), 0),((510,55) , 90),((570,240), -90), ((170,585), 45),((160,565), -135)]
```

```python
def random_track_func():
    finish_index = random.randint(0, len(finish_line_options) - 1)

    (finish_img, finish_mask), finish_pos = finish_line_options[finish_index]
    start_pos, start_angle = random_car_location[finish_index]

    images = [
        (GRASS, (0, 0)),
        (TRACK, (0, 0)),
        (finish_img, finish_pos),
        (TRACK_BORDER, (0, 0)),
    ]

    return images, finish_mask, finish_pos, start_pos, start_angle
```

Esta função é responsável por randomizar o carro e a linha de partida no início da corrida.

3º Etapa

Para gerar o modelo que iria correr, utilizámos várias voltas de corrida feitas por nós com o carro manual. Este ficheiro csv foi preparado usando bibliotecas como o numpy, e pandas.

Como podemos ver em baixo a forma final dos dados engloba as distâncias dos sensores e as ações correspondentes que o jogador efetuou durante a corrida. Uma vez que o carro tinha uma velocidade máxima, as voltas de treino foram praticamente todas feitas com o dedo a pressionar o 'W' o que corresponde a ação acelerar. Numa versão anterior em que só tínhamos uma ação a tabela ficava populada somente com 'accelerate' uma vez que só registava a aceleração e não as curvas. Para evitar isso decidimos adicionar a combinação de ações. Quando necessário o carro registava não só o 'W' mas também as curvas 'A' e 'D' gerando assim ações como 'accelerate+rotRight' permitindo o carro acelerar e virar ao mesmo tempo.

Durante o treino, as voltas foram feitas de forma contínua e estável, tentando sempre conduzir da melhor forma possível. Quando cometia erros, como bater contra a parede, essas voltas eram apagadas e não incluídas no treino da árvore. Isto porque, ao incluir voltas com choques (como testámos posteriormente), o carro treinado passava a encarar o ato de bater como algo normal, reproduzindo esse comportamento durante a corrida. Como isso prejudicava completamente a condução autónoma, decidimos que o melhor seria apenas incluir voltas limpas, sem choques, no processo de treino.

```python
df_laps = pd.read_csv("training data/training_data_6_sensors_1.csv")

mask = (df_laps['action'] == 'none') | (df_laps['action'] == 'accelerate+rotLeft+rotRight')

df_laps = df_laps[~mask]
df_laps
```

	north	north_west	north_east	south	west	east	action	lap
0	100	59	59	100	36	35	accelerate	2
1	100	59	59	100	36	35	accelerate	2
2	100	59	59	100	36	35	accelerate	2
3	100	59	59	100	36	35	accelerate	2
4	100	59	59	100	36	35	accelerate	2
...	...	...	...	...	...	...	...	...
29163	72	42	100	98	34	38	accelerate+rotRight	15
29164	76	42	100	94	33	39	accelerate+rotRight	15
29165	80	44	100	88	33	38	accelerate+rotRight	15
29166	80	43	100	88	33	39	accelerate+rotRight	15
29167	81	43	100	85	32	38	accelerate+rotRight	15

29168 rows × 8 columns

De seguida treinámos os dados inicialmente numa árvore de decisão do scikitLearn e mais tarde também noutros modelos como o Random Forest ou Gradient Boost.
Conseguindo obter resultados a rondar os 80% de *accuracy*.

```python
dtc = DecisionTreeClassifier(criterion ='entropy',splitter='random', max_depth=6, min_samples_leaf = 2, min_samples_
dtc.fit(X_train, y_train)

print('Decision Tree Entropy max depth')
print('Accuracy train:', dtc.score(X_train,y_train))
print('Accuracy test:', dtc.score(X_test,y_test))
```

```
Decision Tree Gini max depth
Accuracy train: 0.8118028890107881
Accuracy test: 0.811848601206802
```

Para ver quais eram os melhores parâmetros para cada csv utilizámos uma Grid Search com cross validation.

Grid Search

```python
from sklearn.model_selection import GridSearchCV

param_grid = {
    'max_depth': [4, 6, 8, 10, None],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
    'criterion': ['gini', 'entropy'],
    'splitter': ['best', 'random']
}

grid = GridSearchCV(DecisionTreeClassifier(), param_grid, cv=10,scoring='accuracy',n_jobs=-1,verbose=2)
grid.fit(X_train, y_train)

print("Best params:", grid.best_params_)
print("Best CV accuracy:", grid.best_score_)

# Evaluate on test set
best_rf = grid.best_estimator_
print("Test accuracy:", best_rf.score(X_test, y_test))
```

```
Fitting 10 folds for each of 80 candidates, totalling 800 fits
Best params: {'criterion': 'entropy', 'max_depth': 4, 'min_samples_leaf': 1, 'min_samples_split': 2, 'splitter': 'best'}
Best CV accuracy: 0.8119411362973329
Test accuracy: 0.8143170597915523
```

4º Etapa

Com o modelo pronto utilizámos o joblib para exportar o modelo para o notebook do carro.

```python
from decisionTrees import *
from sklearn.tree import DecisionTreeClassifier
import joblib  # or pickle if you used that

model = joblib.load("trained_tree_model.pkl")
#le = joblib.load("label_encoder.pkl")  #gradient boost need
```

No carro, alterámos a lógica de modo a conseguir ler as input actions e convertê-las em ações. Para testar o Gradient Boost necessitámos que adicionar um 'conversor' das ações uma vez que o output deste modelo é 0,1,2,3 etc.

```python
# In your PlayerCar class or main loop:
sensor_data = [self.sensors[i] for i in range(len(self.SENSOR_NAMES))]
action = model.predict([sensor_data])[0]  # Get predicted action like "accelerate", "brake", etc.
#predicted_action = le.inverse_transform([action])[0]  xdboost use this and replace bellow with predict_action
# como estamos a usar o XDGradient Boost o seu output é 1,2,3, o carro só lê rotate,accelerate...
```

```python
# Convert to actual action
if action == "accelerate":
    self.accelerate()
elif action == "brake":
    self.brake()
elif action == "rotLeft":
    self.rotLeft()
elif action == "rotRight":
    self.rotRight()
elif action == 'accelerate+rotRight':
    self.accelerate()
    self.rotRight()
elif action == 'accelerate+rotLeft':
    self.accelerate()
    self.rotLeft()
```

5º Etapa

Após termos o carro a funcionar de modo mais ou menos satisfatório, fomos ver o que podíamos fazer para melhorar a condução autónoma do mesmo. Uma das coisas que fizemos foi adicionar mais sensores (10 no total) ao carro, para ver se isso melhorava a sua condução. Após treinar um novo modelo e aplicá-lo ao carro, não verificamos nenhuma melhoria significativa, parecendo até que o desempenho piorou. Infelizmente, não conseguimos melhorar o comportamento do carro em relação ao primeiro modelo, mesmo alterando parâmetros como o número de sensores e os próprios modelos de aprendizagem.

Durante o processo de treino, experimentámos também diferentes formas de controlar o carro manualmente, com o objetivo de fornecer dados mais variados e, teoricamente, mais ricos para o modelo aprender. Numa dessas tentativas, em vez de conduzir com movimentos mais fluidos (por exemplo, fazer uma curva de uma só vez), testei pressionar as teclas com mais frequência durante as voltas. A ideia era aumentar o número de linhas registadas no ficheiro CSV, já que cada ação regista uma entrada, permitindo à árvore de decisão ter mais dados para analisar. No entanto, este método não deu os resultados esperados. O carro deixou de se mover suavemente, o que já era previsível, mas pior que isso, estava constantemente a virar-se sem conseguir andar em linha reta, o que não era suposto, já que durante o treino ajustava sempre a sua posição para o manter a meio da pista, o que impedia que fizesse curvas corretamente ou até seguisse em frente de forma estável. Assim, decidimos voltar ao método original de condução, mais suave e contínuo, por se revelar mais eficaz para o treino do modelo.

6º Etapa

Adicionar um mecanismo de salvaguarda, para quando o carro fica preso.

```python
def step(self,collision=False):
    self.update_sensors()
    valores = {}
    current_time = pygame.time.get_ticks()

    if collision:
        if self.last_collision_time is None:
            self.bounce()
            self.angle += random.randint(-25, 25)
            return
```

Uma vez que a condução do carro está longe de estar perfeita e o facto de quando bate nas paredes adiciona um mecanismo de 'bounce' e entra num loop infinito decidimos adicionar umas salvaguardas.

Apesar de nem sempre funcionar bem o carro, de momento após colidir com a parede, aciona o bounce (-velocidade) e adiciona também uma rotação random entre -25,25 à sua rotação quando bateu. A ideia é tirar o carro do loop adicionando um fator aleatório.

Conclusão

Foi possível obter um carro a conduzir de forma autónoma com resultados razoáveis, tendo por base modelos de aprendizagem treinados com dados recolhidos manualmente. A abordagem inicial, com uma árvore de decisão construída à mão, acabou por ser surpreendentemente eficaz, tendo tido melhores resultados que os modelos treinados posteriormente.

Ao longo do projeto, foi testada a influência de diversos fatores, como o número de sensores, a qualidade dos dados de treino, e o tipo de modelo utilizado. Concluímos que o modelo final é altamente dependente da forma como os dados são recolhidos. Pequenas variações na condução durante o treino afetaram diretamente o desempenho. Tentativas de gerar mais dados por meio de inputs mais frequentes revelaram-se contraproducentes, resultando num comportamento errático do carro.

Mesmo com modelos mais avançados e mais sensores, não se verificaram melhorias significativas. Por isso, mantivemos uma configuração mais simples e voltámos a estratégias de treino que garantiam estabilidade e consistência.

Finalmente, foram adicionadas medidas de salvaguarda para contornar situações onde o carro ficava preso após colisões, embora essa solução ainda apresente limitações. No estado atual, o carro é capaz de percorrer a pista de forma autónoma e consistente, dentro dos limites do sistema desenvolvido e dos dados fornecidos.